

Opcional - LAB: Orquestación de Kubernetes en AWS con Amazon EKS

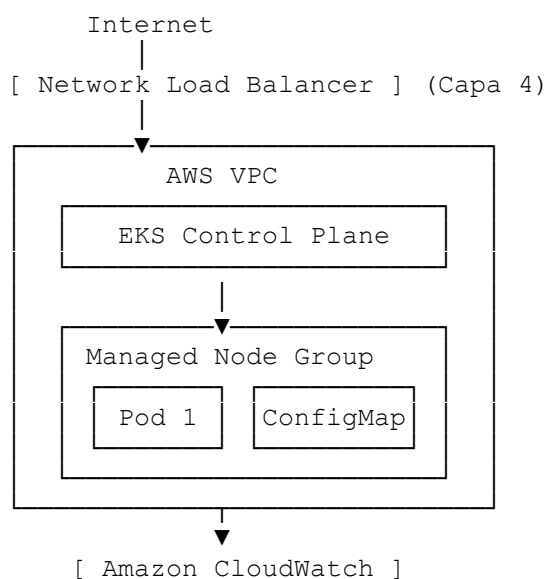
1. Resumen del lab

Este laboratorio "end-to-end" guía al estudiante en el aprovisionamiento, operación y escalado de un clúster de Amazon EKS utilizando herramientas de nivel profesional (`eksctl` y `kubectl`). El ejercicio cubre desde la creación de la infraestructura base, inyección de configuraciones (ConfigMaps), hasta la implementación de capacidades de escalado dinámico (HPA) y observabilidad total mediante CloudWatch Container Insights.

2. Arquitectura objetivo

- **Control Plane:** Amazon EKS (Gestionado por AWS).
- **Data Plane:** Managed Node Group (2 instancias `t3.medium` en subredes privadas).
- **Networking:** VPC dedicada con balanceador de carga de red (NLB).
- **Configuración:** ConfigMap inyectado como volumen para la interfaz web.
- **Observabilidad:** Metrics Server (interno) y CloudWatch Observability (nativo).

Plaintext



3. Objetivos de aprendizaje

1. Aprovisionar un clúster de EKS de forma declarativa con `eksctl`.
2. Solucionar problemas de comunicación interna parcheando el Metrics Server.
3. Desacoplar la configuración del contenedor utilizando ConfigMaps.
4. Configurar el escalado horizontal automático (HPA) con umbrales ajustados.
5. Implementar observabilidad nativa de contenedores en AWS.

4. Prerrequisitos y verificación

- **Cuentas/Roles:** Acceso AWS con permisos de Administrador.
- **Herramientas:** AWS CLI v2, kubectl (v1.28+), eksctl (v0.170+).
- **Verificación rápida:**

```
aws sts get-caller-identity
eksctl version
kubectl version --client
```

5. Setup inicial

Configura la región por defecto en tu terminal para evitar inconsistencias:

```
export AWS_REGION=us-east-1
```

6. Ejecución del lab

Paso 1: Creación del Clúster

Aprovisiona la infraestructura base (VPC, IAM, Nodos). *Toma ~20 minutos.*

```
eksctl create cluster \
  --name k8s-lab-cluster \
  --region $AWS_REGION \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 2 \
  --managed
```

Paso 2: Instalación y Parche del Metrics Server

Instala el recolector de métricas y aplica los parches de seguridad/etiquetas necesarios para EKS.

```
# 1. Aplicar manifiesto
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# 2. Parchear TLS
kubectl patch deployment metrics-server -n kube-system --type='json' -p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'
```

```
# 3. Parchear etiquetas (Soluciona error MissingEndpoints)
kubectl patch deployment metrics-server -n kube-system -p '{"spec": {"template": {"metadata": {"labels": {"k8s-app": "metrics-server"}}}}}'
```

Paso 3: ConfigMap (Interfaz Web Personalizada)

Crea la página web y guárdala en el clúster.

```
cat <<'EOF' > index.html
<!DOCTYPE html>
```

```

<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>EKS Lab - AWS</title>
  <style>
    body { background-color: #1e1e2e; color: #cdd6f4; font-family:
sans-serif; display: flex; flex-direction: column; align-items:
center; justify-content: center; height: 100vh; margin: 0; }
    h1 { color: #89b4fa; font-size: 3rem; }
    .status { margin-top: 30px; padding: 15px 30px; background-
color: #a6e3a1; color: #11111b; border-radius: 8px; font-weight: bold;
font-size: 1.5rem; }
  </style>
</head>
<body>
  <h1>🚀 EKS Cluster Operativo</h1>
  <p>Laboratorio de orquestación y autoescalado completado con
éxito!</p>
  <p>Tráfico balanceado por AWS Network Load Balancer.</p>
  <div class="status">HPA Status: ONLINE</div>
</body>
</html>
EOF

```

```
kubectl create configmap custom-website --from-file=index.html
```

Paso 4: Despliegue de la Aplicación (Generación con --dry-run)

1. Generar y editar el Deployment:

```
kubectl create deployment nginx-deployment --image=nginx:alpine --dry-
run=client -o yaml > deployment.yaml
```

Abre `deployment.yaml` y asegúrate de agregar los recursos (requests) y el montaje del ConfigMap en la sección de contenedores:

YAML

```

# ... (dentro de spec.template.spec)
containers:
- image: nginx:alpine
  name: nginx
  resources:           # 1. AÑADIR LIMITES
    requests:
      cpu: "100m"
      memory: "128Mi"
  volumeMounts:       # 2. MONTAR VOLUMEN
    - name: web-volume
      mountPath: /usr/share/nginx/html
  volumes:             # 3. DEFINIR VOLUMEN
    - name: web-volume
      configMap:
        name: custom-website

```

2. Generar y editar el Servicio (NLB):

```
kubectl create service loadbalancer nginx-service --tcp=80:80 --dry-
run=client -o yaml > service.yaml
```

Abre service.yaml, corrige el selector y añade la anotación de AWS:

YAML

```
metadata:
  annotations: # AÑADIR ANOTACIÓN
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
spec:
  selector:
    app: nginx-deployment # ASEGURAR QUE COINCIDA CON EL DEPLOYMENT
```

3. Aplicar al clúster:

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Paso 5: Autoescalado (HPA) y Prueba de Estrés

1. Crear el HPA con un umbral del 30%:

```
kubectl autoscale deployment nginx-deployment --cpu-percent=30 --min=1
--max=5
```

2. Script de Estrés: Crea este script (stress.sh) para saturar la red localmente y disparar el HPA:

```
cat <<'EOF' > stress.sh
#!/bin/bash
URL=$1
echo "Atacando $URL con 50 hilos..."
for i in $(seq 1 50); do
  while true; do curl -s -o /dev/null "$URL"; done &
done
wait
EOF
chmod +x stress.sh
```

3. Ejecutar y Monitorear: Obtén la URL de tu balanceador (kubectl get svc nginx-service) y lanza el ataque:

```
./stress.sh http://<TU_URL_AQUÍ>
```

En otra terminal, monitorea el escalado: kubectl get hpa -w.

Paso 6: Observabilidad con CloudWatch

```
eksctl create addon --name amazon-cloudwatch-observability --cluster
k8s-lab-cluster --force
```

7. Retos opcionales

1. **Actualización In-Place:** Modifica el mensaje dentro de tu index.html, actualiza el ConfigMap y fuerza a los pods a recargar el volumen.
2. **Node Affinity:** Configura el deployment para que solo se ejecute en una zona de disponibilidad específica.

8. Troubleshooting

- **Error localhost:8080:** Expiración de credenciales. Ejecuta `aws eks update-kubeconfig --region us-east-1 --name k8s-lab-cluster`.
- **HPA estancado en <unknown>:** Falta el bloque `resources.requests` en el `deployment.yaml` o el Metrics Server no está funcionando.
- **El NLB no carga en el navegador:** Los Network Load Balancers de AWS tardan ~3-5 minutos en registrar los nodos. Sé paciente.

9. Checklist de validación

- [] `kubectl top pods` devuelve métricas de CPU/RAM.
- [] Ingresar el DNS del NLB en el navegador muestra la página oscura "EKS Cluster Operativo".
- [] Ejecutar el script `stress.sh` hace que la columna REPLICAS del HPA suba a 2 o más.

10. Cleanup (OBLIGATORIO)

Previene cargos en tu cuenta de AWS eliminando los recursos en este orden:

Bash

```
kubectl delete hpa nginx-deployment
kubectl delete svc nginx-service
kubectl delete deployment nginx-deployment
kubectl delete configmap custom-website
# Espera 1 minuto antes de ejecutar el último comando
eksctl delete cluster --name k8s-lab-cluster --region us-east-1
```

11. Preguntas de repaso

1. ¿Cuál es el beneficio principal de inyectar una página web a través de un `ConfigMap` en lugar de compilarla dentro de la imagen Docker?
2. ¿Por qué el HPA requiere estrictamente que se definan los `requests` de CPU en el contenedor?
3. En la prueba de estrés, ¿por qué fue necesario bajar el `averageUtilization` al 30% para forzar el escalado?
4. ¿Qué función cumple la anotación `aws-load-balancer-type: "nlb"` en el Servicio?
5. Si el API Server reporta `MissingEndpoints` para un Servicio, ¿cuál es la causa más probable?

Pre -requisitos

1. Instalar `kubectl` (Última versión estable)

El siguiente bloque de comandos consulta dinámicamente la API de versiones de Kubernetes para descargar el último binario, valida su integridad y lo instala en tu sistema.

```
# 1. Descargar el binario de la última versión estable
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1"

# 2. Descargar el checksum y validar el archivo (Buena práctica de
seguridad)
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1.sha256"
echo "$(cat kubect1.sha256) kubect1" | sha256sum --check

# 3. Instalar el binario con los permisos correctos en el PATH del
sistema
sudo install -o root -g root -m 0755 kubect1 /usr/local/bin/kubect1

# 4. Limpiar los archivos descargados
rm kubect1 kubect1.sha256
```

(Expected output del paso 2: kubect1: la suma coincide o kubect1: OK)

2. Instalar eksctl (Última versión)

La herramienta eksctl se distribuye oficialmente a través de GitHub Releases. Descargaremos el archivo comprimido, lo extraeremos y lo ubicaremos junto a kubect1.

```
# 1. Definir variables de arquitectura y plataforma
ARCH=amd64
PLATFORM=$(uname -s)_$ARCH

# 2. Descargar el archivo tar.gz de la última versión
curl -sLO "https://github.com/eksctl-
io/eksctl/releases/latest/download/eksctl_$PLATFORM.tar.gz"

# 3. Extraer el binario en el directorio temporal
tar -xzf eksctl_$PLATFORM.tar.gz -C /tmp

# 4. Mover el binario al PATH del sistema y limpiar el temporal
sudo mv /tmp/eksctl /usr/local/bin/
rm eksctl_$PLATFORM.tar.gz
```

3. Verificación del entorno

Para confirmar que ambas herramientas están correctamente instaladas y listas para usar, ejecuta estos comandos de validación:

```
kubect1 version --client --output=yaml
# Expected: Debería mostrar un bloque YAML con "clientVersion" y la
arquitectura linux/amd64.

eksctl version
# Expected: Debería imprimir la versión actual, por ejemplo: 0.17x.x
```